

dilemma Artifact Evaluation Instructions

Introduction

This artifact includes instructions for running a docker image which contains our implementation of our lemma synthesis technique, as well as all of the tests that we ran and scripts/instructions to run them. The artifact includes a script that compiles all of the tests and generates the lists of lemmas that were synthesized.

There are 7 sets of benchmarks we considered: VFA, clam (atomic/implications), lia clam (atomic/implications), fulladder, and compiler. We'll list the breakdowns of the successes and failures for those sets. Instructions/information for how we categorized a result to be a success, found a useful lemma or fail are described in greater detail within the step-by-step evaluation portion below. The instructions for running VFA benchmarks are a bit different than the others just because of modifications to artifact and setup.

Evaluation Benchmarks:

- VFA Benchmarks (226): 77 Successes, 20 Usefals, 129 Failures

Benchmarks Exactly Matching Prior Work:

- Clam (atomic) Benchmarks (140): 54 Successes, 86 Failures
- Clam (implication) Benchmarks (18): 9 Successes, 9 Failures
- Lia (atomic) Benchmarks (19): 6 Successes, 13 Failures
- Lia (implication) Benchmarks (9): 4 Successes, 9 Failures
- Fulladder Benchmarks (62): 7 Successes, 55 Failures
- Compiler Benchmark (1): 0 Successes, 1 Failure

There are about 23 hours worth of tests to run, we've broken these down into smaller groups so that you can run some and come back to others later. For the benchmarks that match prior work, we've grouped them together by which we expect to succeed vs fail. This is done to limit the time you have to run experiments if you would like to skip running the experiments which we expect to fail. The evaluation of the results requires some manual effort (looking at the top 5 lemmas returned and comparing them to the target). There are more instructions and details on these later on in the document.

Hardware Requirements

There shouldn't be any special hardware requirements. All of our experiments were run on MacOS using an Apple M1 Pro chip with 10 cores and 16GB memory. Our tool does run multiple processes concurrently, but there are no requirements. We use as many cores that are

available and if there are none, then the code is run linearly. We expect any laptop or computer that any evaluator has should work.

Getting Started

Setting up Artifact

Download the file `dilemma-docker.tar` that is provided. The following commands will start-up the virtual environment, see the following sections for exact commands to run to evaluate:

```
$ docker load -i {...}/dilemma-docker.tar
$ docker run -ti -v ${PWD}/results:/root/dilemma-artifact/results
dilemma-artifact

                                <starts environment>
$ root@bf512e7c455b:~/dilemma-artifact# bash make_all.sh
$ root@bf512e7c455b:~/dilemma-artifact# eval $(opam config env)
```

At this point, the environment is set up to run. The “`make_all.sh`” script ensures that all the dependencies for the tests are in place and installed. Finally, “`eval $(opam config env)`” ensures that the command line is able to find all path variables (specifically, for our evaluation that means the command “`coqc`” is available). The following sections will detail experiment setup and evaluation instructions.

Note, all of the results from the evaluations are expected to be found in the `${PWD}/results` directory as specified in the run command. That is, after running the container there should be a directory created in that same directory titled ‘`results`’.

Note: Ensure that you run the `docker run` command outside of your downloads folder. In general, there are some folders that docker is not able to modify (in our case, add a results folder). We’ve had success running the command from the desktop folder or home folder on your local machine.

Simple Test (Kick-the-Tires)

There are more details in the experiment set up – specifically, with respect to what the benchmarks are and how to evaluate the results. In this section, we’ll make sure you can run a single group and a single test suite. The remaining portion of the artifact evaluation will entail running the remaining groups/suites and then reviewing the results.

There are 14 sets of benchmarks that we are currently evaluating on. For each of the benchmark suites, we've broken them into groups where each group should take approximately 20 minutes to run.

It is possible that some of the runs will print out errors and in turn there will be no results in the results file generated for that test. Your docker setup is still fine if this is the case. It is likely the case that because our tools require writing and compiling external files that the docker results in segmentation faults and writing issues that don't frequently occur when run locally. There are comments on this in the later sections of the instructions. Including this here so that if you see errors getting printed out when running, that isn't necessarily unexpected.

Run a VFA Benchmark Suite: Try running the bagperm benchmark suite. Following the set up listed above, run the following command:

```
$ root@bf512e7c455b:~/dilemma-artifact# bash run.sh bagperm
```

Benchmark Suite Result: Following the command above, there should be a file titled "suite_bagperm.txt" found in the results folder that is generated from the `docker run` command. This file should include various outputs for each test (we'll explain how to interpret these results later in this document) that looks something like this:

```
Test: bag_perm_by_bag_eqv_uncons
Target: bag_eqv (b :: l1) (b :: l2) -> bag_eqv l1 l2

(bag_eqv (n :: a1) (n :: b1) -> bag_eqv a1 b1)
```

```
Number of Result Pairs Returned (before QuickChick): 1
```

```
Time Elapsed From Start: 29.164 seconds
```

```
Test: bag_perm_by_count_insert_other
Target: y <> x -> count y (l1 ++ x :: l2) = count y (l1 ++ l2)

(count n0 (a :: a1) = count n0 (n :: x ++ a :: x0) -> count n0
a1 = count n0 ((n :: x) ++ x0))
```

```
Number of Result Pairs Returned (before QuickChick): 1
```

```
Time Elapsed From Start: 102.255 seconds
```


...file continues...

Run a VFA Benchmark Group: Try running the first selection benchmark group (merge_3). Following the set up listed above, run the following command:

```
$ root@bf512e7c455b:~/dilemma-artifact# bash run.sh group merge_3
```

VFA Benchmark Group Result: Following the command above, there should be a file titled “group_merge_3.txt” found in the results folder that is generated from the `docker run` command. This file should include various outputs similar to one above.

Run an VFA Update Benchmark Group: There is an additional option to run with smaller groups, in order to have small runtime sections. Try running the first selection benchmark group (selection_3). Following the set up listed above, run the following command:

```
$ root@bf512e7c455b:~/dilemma-artifact# bash run.sh group_updated  
selection_3
```

Benchmark Group Result: Following the command above, there should be a file titled “group_updated_selection_3.txt” found in the results folder that is generated from the `docker run` command. This file should include various outputs similar to one above.

Run a Prior Work Benchmark Group: These instructions are for running the remaining benchmarks. Try running a set of the benchmarks from the lia atomic benchmarks that succeeded (say lia_successes_1). Following the set up listed above, run the following command:

```
$ root@bf512e7c455b:~/dilemma-artifact# bash run.sh lfind_benchmarks  
lia_successes_1
```

Prior Work Benchmark Group Result: Following the command above, there should be a file titled “group_lfind_lia_successes_1.txt” found in the results folder that is generated from the `docker run` command. This file should include various outputs similar to one above.

If you are able to run these commands and view the corresponding result files in the results folder, you should be good to run all of them. We would suggest if you take breaks between running to make sure you have the results saved in a different directory (in order to prevent any overwriting of files and needing to rerun test sets).

Experiment Setup

There are 16 sets of benchmarks that we are evaluating on – 10 from the VFA set and 6 from prior works. The table below details the VFA benchmarks :

Benchmark Suite	Number of Test Locations	Number of ~20 min Groups
bagperm	11	1
binom	46	16
merge	17	3
perm	1	1
priqueue	8	1
redblack	32	8
searchtree	59	13
selection	24	2
sort	11	1
trie	17	1

We've provided two means of running the tests. The first option is to run a benchmark suite at a time. In many cases this is probably easiest, however, there are a few larger benchmarks that we have divided into groups to make it easier to run groups at a time and come back later. The rightmost column details how many groups that benchmark was divided into.

Note, that we cut off each group at ~20 minutes, however this was done greedily. So, there are cases where the full 20 minutes is not taken, and in cases where a single test might take longer than 20 minutes that group will just hold that single test. The different groups from each benchmark and their expected runtimes are listed below:

Benchmark Suite	Group Labels	Expected Runtime (minutes)
bagperm	bagperm	12
binom	binom_1	20
	binom_2	20
	binom_3	20
	binom_4	16
	binom_5	21

	binom_6 binom_7 binom_8 binom_9 binom_10 binom_11 binom_12 binom_13 binom_14 binom_15 binom_16	19 19 38 18 20 6 17 38 15 29 4
merge	merge_1 merge_2 merge_3	16 20 7
perm	perm	1
priqueue	priqueue	18
redblack	redblack_1 redblack_2 redblack_3 redblack_4 redblack_5 redblack_6 redblack_7 redblack_8	11 22 16 20 21 15 18 19
searchtree	searchtree_1 searchtree_2 searchtree_3 searchtree_4 searchtree_5 searchtree_6 searchtree_7 searchtree_8 searchtree_9 searchtree_10 searchtree_11 searchtree_12 searchtree_13	19 19 19 19 6 17 17 13 21 13 19 17 10
selection	selection_1 selection_2	20 9
sort	sort	12

trie	trie	18
------	------	----

There is an additional group partition which has smaller runtimes than those listed above. Each of the groups has the following number of groups. To run an updated group, run:

```
$ root@bf512e7c455b:~/dilemma-artifact# bash run.sh group_updated
{benchmark}_{updated group label}
```

Benchmark Suite	Updated Group Labels Range
bagperm	3
binom	23
merge	5
perm	1
priqueue	2
redblack	16
searchtree	24
selection	6
sort	3
trie	5

For example, the trie benchmark set has five updated group labels. To run the third group, do:

```
$ root@bf512e7c455b:~/dilemma-artifact# bash run.sh group_updated
trie_3
```

Finally, the following table details the groups to run the remaining benchmarks from prior works, and is followed by instructions. Note, clam_implications_failures_[1,2] can be interpreted as groups clam_implications_failures_1 and clam_implications_failures_2.

Benchmark Suite	Group Names
clam (atomic)	clam_failures_[1,2,...,8] clam_successes_[1,...,14]
clam (implication)	clam_implications_failures_[1,2] clam_implications_successes_[1,2]

lia (atomic)	lia_failures_[1,2,3] lia_successes_[1,2]
lia (implication)	lia_implications_failures lia_implications_successes
fulladder	fulladder_failures_[1,...,14] fulladder_successes_[1,2]
compiler	compiler

To run one of the groups use the following commands:

```
$ root@bf512e7c455b:~/dilemma-artifact# bash run.sh lfind_benchmarks
{group name}
```

For example, to run the third group of the clam_success, use the following command:

```
$ root@bf512e7c455b:~/dilemma-artifact# bash run.sh lfind_benchmarks
clam_successes_3
```

Each benchmark suite or group contains a set of files, each file is a Coq file which includes a partial proof where the proof is ended with a call to our tactic (`dilemma. Admitted.`).

To run each test individually, one can call into the directory and compile the file. For example to run the test `select_rest_length_by_select_perm.v` from the selection benchmark suite, one would run:

```
$ cd dilemma-artifact/benchmarks/vfa_selection/tests
$ coqc select_rest_length_by_select_perm.v
```

The results will be available printed out to the terminal (and the whole log can be seen in the same folder in a folder titled

“log_for_select_rest_length_by_select_perm1.txt”).

For the scripts with the “lfind_benchmarks” (in red), if you want to run an individual test, the paths are a bit more specific (in order to match exactly). The script will print out the full path to the terminal so you can easily get the full files if you need to rerun (or the file `dilemma-artifact/testing_scripts/groups_lfind.json` holds the files associated with each of the groups). For example, suppose the path printed is:

“benchmarks/lia/atomic/queue_len_by_len_app.v.”

To run the individual test the same way as listed above:

```
$ cd dilemma-artifact/benchmarks/lia/atomic
$ coqc queue_len_by_len_app.v
```


**** NOTE:** Sometimes when running tests back to back, the temporary files we write get messed up due to space issues (especially in the docker image). Oftentimes if you rerun the experiment it will work. In these cases if there are no results in the log after running (that is just the test and target lemma are listed with no info), you can either rerun the tests that didn't complete in each group individually or let us know a list which tests didn't complete and we can make another group of those tests to rerun (and push those changes to the repo). If rerunning again fails, then the case should be marked as failure.

To test individually, just move to the directory where that test is located and compile that file. For example, suppose the test `t.v` from the benchmark set `benchmark_suite` needs to be reran you can run the following commands:

```
$ root@bf512e7c455b:~/dilemma-artifact# cd benchmarks/benchmark_suite/tests
$ ... :~/dilemma-artifact/benchmarks/benchmark_suite/tests# coqc test.v
```

The response that matches the output found in the results file should be printed to the terminal (that is the top 5 suggested lemmas).

This should not occur too frequently - although it is likely that running the docker image will cause these issues to be more common. Let us know if the issues persist and we can break the groups down more so the tests are less cumbersome and/or make groups of tests that need to be rerun so we can put them all under one group label to make it easier. Additionally we can provide more detailed instructions for locally running the benchmarks using the Github link from the Zendo. The installation is all within an opam switch so is fairly simple.

For reference, there were only a few cases that needed to be rerun in the VFA benchmarks during our evaluation, and a bit more in the lia/clam benchmarks due to search complexity of those benchmarks. In total, there were no more than 20 that needed to be rerun to run successfully. Note, this was stated about running the tool for our testing and evaluation, not in the docker. It is likely there will be more cases that need to be rerun in the docker image than if run locally.

Step-by-Step Instructions (Evaluation Instruction)

Generate Results

[VFA] To run the whole benchmark suite at time run:

```
$ root@bf512e7c455b:~/dilemma-artifact# bash run.sh <benchmark>
```

Where `<benchmark>` is the name of the benchmark suite to run (should correspond to a row in the first table listed in this document). For example, to run the redblack benchmark suite, you would run:

```
$ root@bf512e7c455b:~/dilemma-artifact# bash run.sh redblack
```

The `results` folder will hold the results from this benchmark suite in a file called “`suite_redblack.txt`”. The same is expected for running any of the suites (where file is labeled `suite_<suite name>.txt`).

[VFA] To run the a single group of benchmarks at time run:

```
$ root@bf512e7c455b:~/dilemma-artifact# bash run.sh group <group>
```

Where `<group>` is the name of the group of benchmarks to run from VFA (this should be an element from the middle column of the second table listed above). For example, to run the second group from the merge benchmark suite, you would run:

```
$ root@bf512e7c455b:~/dilemma-artifact# bash run.sh group merge_2
```

The `results` folder will hold the results from this benchmark suite in a file called “`group_merge_2.txt`”. The same is expected for running any of the groups (where file is labeled `group_<group name>.txt`).

[Prior Work] To run the a single group of benchmarks at time run:

```
$ root@bf512e7c455b:~/dilemma-artifact# bash run.sh lfind_benchmarks  
<group>
```

Where `<group>` is the name of the group of benchmarks to run from the prior work benchmarks (this should be an element from the right column of the fourth table listed above). For example, to run the second group from the clam atomic benchmarks suite that succeeded, you would run:

```
$ root@bf512e7c455b:~/dilemma-artifact# bash run.sh lfind_benchmarks  
clam_successes_2
```

The `results` folder will hold the results from this benchmark suite in a file called “`group_lfind_clam_successes_2.txt`”. The same is expected for running any of the groups (where file is labeled `group_<group name>.txt`).

[VFA] To run the a single test:

```
$ root@bf512e7c455b:~/dilemma-artifact# cd  
dilemma-artifact/benchmarks/{benchmark suite}/tests  
$ root@bf512e7c455b:~/dilemma-artifact# coqc {test to run}.v
```

This will print the top 5 lemmas for that test to the terminal. To see the target lemma for that test, you can look in the log `dilemma-artifact/testing_scripts/target_lemmas.json`

[Prior Work] To run a single test from the prior work benchmarks (lfind benchmarks), you can get the file paths for the tests from the file `dilemma-artifact/testing_scripts/groups_lfind.json`. To run, call into the directory of the file and use `coqc` to compile the file. A log will be generated as described above.

All results will be generated once all of the benchmark suites have been run. This will either happen by running the benchmark suite as instructed above, or by running all of the groups found within that benchmark suite.

The results generated are the lemmas that we've synthesized – the following section details how these results should be interpreted/evaluated.

Interpret Results

The snip below is from one of the result files:

```
...
-----
-----
Test: select_rest_length_by_select_perm
Target: select x l = (y, r) -> Permutation (x :: l) (y :: r)

(select x l = (y, r) -> length l = length r)

(select x l = (y, r) -> Permutation (y :: r) (x :: l))
(Permutation (y :: r) (x :: l) -> length l = length r)

(select x l = (y, r) -> Permutation (x :: l) (y :: r))
(Permutation (x :: l) (y :: r) -> length l = length r)

Number of Result Pairs Returned (before QuickChick): 3

Time Elapsed From Start: 58.977 seconds
-----
-----
...
```

Each result file will have one of these sections for each test that is run. The first line indicates the file that the test is stored in, and the second line represents the helper lemma that was used at the location. The following lines are the lemmas that we've synthesized (this will be restricted to the

top 5 for each test). The following two lines are not relevant for evaluation (one notes the amount of results generated and the other is the runtime).

In the case no lemmas or statistics are returned, that means that the test didn't finish running. Sometimes there are resource issues that occur from running a bunch of tests in a row, you can rerun these tests (described above).

We expect all of the VFA benchmarks to terminate (average of about 3.5 minutes per test). From the remaining benchmarks, they are partitioned into “success” and “failure” groups where we expect the “success” groups to terminate with a lemma that is an exact match to the target lemma in the top 5. “Failure” cases might either take too long or just not return a useful/matching lemma – either way, these tests were not counted as successes by our chosen metric.

In order to interpret the results, you'll need to manually look at each result. The potential categories a test might fall under are:

- Success – we synthesized the target lemma
 - An exact match to target (variables most likely differ)
 - An exact match to target modulo reflexivity
 - Implication has a precondition which is an equality and if that was written in the goal of the implication and that matches the target lemma, then this is also a success. Suppose the target lemma was `Permutation (x :: l) (y :: r)` and we generated `a = x :: l → Permutation a (y :: r)`, this would be counted.
- Useful – we synthesized a useful lemma
 - A lemma found is used later in the same proof, so we found a lemma that was used at some point to complete the proof. This is considered useful. This requires looking at the file to see what lemmas were used later in the proof.
 - Successfully found one precondition, but still need to weaken other assumptions to match exactly. For example, suppose the target lemma is $A \rightarrow B \rightarrow C$ and we synthesize $A \rightarrow D \rightarrow C$. This is deemed useful because we've successfully isolated one needed precondition (weakened one assumption).
 - We've found a lemma that is weaker than the target lemma but is able to be used in the same way. For example, if the target lemma is $\text{In } x \text{ } l \vee \text{In } x \text{ } m \rightarrow \text{In } x \text{ } (l ++ m)$ and we find $\text{In } x \text{ } l \rightarrow \text{In } x \text{ } (l ++ m)$, where is lemma can be used instead of the stronger target lemma. This is considered useful.
- Fail – test is out of scope, no results generated, or no useful results are generated

We've included the logs from the evaluations that we ran for our paper in the artifact as well. Within the directory `paper_results/processed_results`, there are files for each of the benchmarks containing the categorization we've assigned. These contain the same

information from the results you should have generated (some of the notation is slightly different). These files also include all of the categorization notes that we've included from our analysis.

All of the other folders within `paper_results` hold the raw files that are generated as a byproduct of running our tool. These files should be generated in the docker container as you're running the tests - these will not be moved to the results folder, so if you want to look at them you'll do so through the docker container environment and look in the folders where the tests are stored. To complete the evaluation, there shouldn't be a need to look at these files but you can if you are curious.

Expected Results

As cited in our paper, we expect the following breakdown of results...

VFA Benchmarks (226)

- 77 Successes
- 20 Usefals
- 129 Failures

Lia (Atomic) Benchmarks (19)

- 6 Successes
- 13 Failures

Clam (Atomic) Benchmarks (140)

- 54 Successes
- 86 Failures

Fulladder Benchmarks (62)

- 7 Successes
- 55 Failures

Compiler Benchmarks (1)

- 0 Successes
- 1 Failures

Lia (Implication) Benchmarks (9)

- 4 Successes
- 5 Failures

Clam (Implication) Benchmarks (18)

- 9 Successes
- 9 Failures

Note, there is a bit of non-determinism in our procedure for example generation, so it is possible that the results won't match exactly. For example, if insufficient examples were generated or some resource issue occurs, reducing the proof state and/or synthesizing the preconditions might not behave as expected.

Additionally, as mentioned above, there might be cases where a segmentation fault occurs. This didn't happen when running locally (not in a docker) but occurs in usually a test or two per a group (depending on the size). You should be able to run the test individually and it should hopefully work.

Advice for Completing

The total runtime expected to run all of the tests is about 23 hours. We've broken it down so you can do a few chunks at a time and just let it run in the background of whatever you're doing. You can also obviously analyze the results individually before they are all done.

Note, if you run a suite or a group and stop running before it has finished, you won't be able to see the results since they are printed out in batches. This is why we had opted to break the groups down so that you can do it incrementally. If it would help to have them broken down more, let us know and we can add more groups for you to run.

Changes Expected From Revisions [DONE]

There are two changes that might occur/are expected to occur in the artifact resulting from the revisions suggested by reviewers.

1. More tests will be added. Specifically, we expect that two benchmark suites will be added (each potentially divided into an implication and atomic lemma group).
- ~~2. Change to implementation to improve the runtime. Any changes made at this point would be made to improve the runtime and will not be made if any substantial changes to results are caused by the change.~~

Reusability Guide

Our artifact is a Coq tactic. The only requirements beyond the installation requirements that are needed to use our tool are proofs of decidability for any data types and propositions that are in the scope of the proof.

In the docker image, the folder `examples` contains two examples of our tactic being invoked. The files `selection\_e1.v` and `selection\_e2.v` both showcase the tactic being used. This is the `selection\_e1.v` file:

```
From Dilemma Require Import Dilemma.

Require Import dilemma_testing.Definitions.
Require Import dilemma_testing.Decide.

(* These specify the libraries of functions that should be considered during synthesis that
   | are not defined within the above libraries. *)
Require Import Coq.Lists.List.
Require Import Coq.Sorting.Permutation.

Theorem main : forall (x y : list nat) (n m : nat), select n x = (m,y) -> length x = length y.
Proof.
  | intros.
  | dilemma.
  | Admitted.
```

The file “dilemma_testing.Definitions” includes the relevant definitions for the proof. The file “dilemma_testing.Decide” contains the proofs of decidability for the propositions and equality decidability for types. We list `Require Import` with the modules whose definitions we want to be considered in our synthesis (that are not defined in the other imported modules). Note, we only consider definitions that are defined in the imported modules not imported into those modules.

In order to invoke our tool, you just call the `dilemma` tactic, followed by admitting the proof and then compile the file with `coqc`.

Besides runtime, the main limitation of our tactic is the requirement of needing proofs of decidability. While in practice, these are somewhat routine and can often be automatically found; this is not always the case.